

 DAY 28:
DEBOUNCING &
THROTTLING IN
JAVASCRIPT



Lakshmiprasanna Vakada

Optimizing performance is key when handling frequent user interactions like scrolling, resizing, or typing. Today, we'll learn Debouncing & Throttling, two essential techniques for improving efficiency in JavaScript. ⚡

◆ Problem: Expensive Function Calls

Imagine a search bar that triggers an API request every time a user types a letter. This leads to:

- ◆ Unnecessary API calls
- ◆ Performance issues
- ◆ Slower user experience

We can control the execution rate of such functions using debouncing and throttling.

Debouncing

Debouncing ensures a function executes only after a specified delay from the last time it was called.

✓ Best For:

- ✓ Handling user input (e.g., search box typing)
- ✓ Window resizing
- ✓ Preventing excessive API calls

◆ How Debouncing Works

1. The function waits for the user to stop triggering the event.
2. If triggered again, the timer resets.
3. The function runs only after the delay has passed.

◆ Example: Search Input with Debouncing

```
function debounce(func, delay) {  
  let timer;  
  return function (...args) {  
    clearTimeout(timer);  
    timer = setTimeout(() => func.apply(this, args),  
delay);  
  };  
}  
// Function to be debounced  
function fetchSearchResults(query) {  
  console.log("Fetching results for:", query);  
}  
// Debounced function  
const debouncedSearch = debounce(fetchSearchResults,  
500);  
  
// Event listener  
document.getElementById("search").addEventListener("input", (e) => {  
  debouncedSearch(e.target.value);  
});
```

- ✓ Effect: The function executes only after 500ms of no typing.

Throttling

Throttling ensures a function runs at most once per specified time interval, ignoring extra calls within that time.

✓ Best For:

- ✓ Handling scroll events
- ✓ Click event listeners
- ✓ Resizing the window

◆ How Throttling Works

1. The function runs immediately when triggered.
2. It ignores repeated calls until the time interval has passed.
3. Ensures the function executes at a controlled rate.

◆ Example: Scroll Event with Throttling

```
function throttle(func, limit) {  
  let lastCall = 0;  
  return function (...args) {  
    const now = Date.now();  
    if (now - lastCall >= limit) {  
      lastCall = now;  
      func.apply(this, args);  
    }  
  };  
}  
  
// Function to be throttled  
function onScrollEvent() {  
  console.log("Scroll event triggered:",  
Date.now());  
}  
  
// Throttled function  
const throttledScroll = throttle(onScrollEvent,  
1000);  
  
// Event listener  
window.addEventListener("scroll", throttledScroll);
```

✔ Effect: The function executes at most once every 1000ms while scrolling.

🎯 Debounce vs. Throttle – When to Use?

Feature	Debouncing ⌚	Throttling 🚀
Concept	Delays function execution until inactivity	Limits function execution to intervals
Best For	Search input, resizing events	Scroll, mouse movement, API polling
Execution	Runs after the user stops triggering events	Runs at a fixed interval